

PROGRAMMING WITH C++

UNIT 6

EXCEPTION HANDLING

Outlines

- Introduction to Exception
- Exception Example
- Multiple Exceptions
- Rethrowing an Exception
- Stack Unwinding
- Constructors, Destructors and Exception Handling

Introduction to Exception

- Exception handling is a powerful mechanism in C++ that allows programmers to handle runtime errors or unexpected events gracefully. It provides a structured way to deal with errors, ensuring that the program can recover from abnormal conditions without crashing or producing undefined behavior.

What is an Exception?

- An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. These events are typically caused by:
 - Invalid input (e.g., dividing by zero).
 - Resource unavailability (e.g., file not found, memory allocation failure).
 - Logical errors in the code (e.g., accessing out-of-bounds array elements).

When such an event occurs, the program can "throw" an exception, signaling that something unusual has happened. The exception-handling mechanism then takes over to manage the situation.

Why Use Exception Handling?

- Without exception handling, error management often relies on return codes, conditional checks, and global error flags. This approach can lead to:
- **Code clutter** : Error-checking logic is scattered throughout the program.
- **Reduced readability** : The main logic of the program gets obscured by repetitive error-handling code.
- **Error propagation issues** : Errors may go unnoticed if not properly propagated up the call stack.

Exception handling addresses these problems by:

- Separating error-handling code from the main logic.
- Allowing errors to propagate automatically through the call stack.
- Providing a centralized mechanism for dealing with errors.

Key Concepts in Exception Handling

- **Throwing an Exception:** When an error occurs, the program explicitly **throws** an exception using the throw keyword.
- **Catching an Exception:** A block of code designated to handle exceptions uses the **try** and **catch** keywords to "catch" and process the exception.
- **Exception Propagation:** If an exception is not caught in the current scope, it propagates up the call stack until it is handled or the program terminates.

Key Terminology

Term	Meaning
try	Block where exception might occur
throw	Keyword to signal an exception
catch	Block that handles the exception
Exception	An object that contains information about the error

Syntax

```
#include <iostream>
using namespace std;

int main() {
    try {
        // Code that may throw an exception
        throw "An exception occurred!";
    }
    catch (const char* msg) {
        // Handling the exception
        cout << "Caught exception: " << msg << endl;
    }
    return 0;
}
```

Example Scenario

- Imagine a function that divides two numbers. If the divisor is zero, the function cannot proceed normally. Without exception handling, you might return an error code or use a global flag to indicate failure. With exception handling, you can throw an exception when the divisor is zero, allowing the calling code to handle the error appropriately.


```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 0;

    try {
        if (b == 0)
            throw "Division by zero error!";
        cout << "Result: " << a / b << endl;
    }
    catch (const char* msg) {
        cout << "Exception caught: " << msg << endl;
    }

    return 0;
}
```

Output:

```
Exception caught: Division by zero error!
```

Throwing an Exception

An exception is thrown using the `throw` keyword followed by an exception object.

cpp

 Copy

 Edit

```
throw 10; // Throwing an integer
throw "File not found"; // Throwing a string
throw runtime_error("Runtime error occurred"); // Throwing an exception object
```

Catching an Exception

Catching an exception is done using the `catch` block, which takes the thrown exception as an argument.

cpp

 Copy

 Edit

```
try {
    throw 100;
}
catch (int e) {
    cout << "Caught exception with value: " << e << endl;
}
```

Types of Exceptions in C++

1. Synchronous Exceptions

- These occur **during the program execution** due to issues that can be predicted and handled.
- Examples:
 - Division by zero
 - File not found
 - Invalid array index
 - Memory allocation failure
- ◆ Handled using try, catch, and throw statements.

2. Asynchronous Exceptions

- These occur **outside the flow of the program** and are usually related to external events.
- Examples:
 - Hardware failure
 - Power failure
 - Keyboard interrupt (in some systems)
- **Note:** C++ does not natively support asynchronous exception handling — such exceptions are usually handled by the operating system or using signal handling libraries.

Benefits of Exception Handling

- **Improved Robustness:** Programs can handle errors gracefully instead of crashing.
- **Separation of Concerns:** Normal program logic is separated from error-handling logic.
- **Flexibility:** Exceptions can be used to handle a wide variety of error types, from simple issues to complex scenarios.
- **Standardization:** C++ provides a standardized mechanism for exception handling, making code more maintainable and portable.

Handling Multiple Exceptions in C++

- In real-world applications, a program may encounter various types of errors or exceptional conditions. C++ allows you to handle multiple exceptions by using multiple catch blocks. Each catch block can be designed to handle a specific type of exception, ensuring that the appropriate response is executed for each error scenario.

Why Handle Multiple Exceptions?

- When writing robust software, it's common to anticipate and handle different types of errors. For example:
- A file operation might fail due to "file not found" or "permission denied."
- A mathematical operation might fail due to "division by zero" or "overflow."
- A memory allocation might fail due to "out of memory."

Handling multiple exceptions allows you to:

- Provide tailored responses for different error types.
- Maintain clarity and organization in your error-handling code.
- Ensure that all potential errors are addressed appropriately.

Syntax for Handling Multiple Exceptions

```
try {  
    // Code that may throw different types of exceptions  
}  
catch (int e) {  
    // Handle integer exception  
}  
catch (const char* msg) {  
    // Handle string exception  
}  
catch (...) {  
    // Catch-all handler for any other types  
}
```

Key Points

When an exception is thrown, the program searches for the first `catch` block that matches the type of the thrown exception.

The matching process follows these rules:

1. **Exact Match** : The exception type matches the type specified in the `catch` block.
2. **Base Class Match** : If the exception is derived from a base class, a `catch` block for the base class can handle it.
3. **Catch-All Block** : The `catch (...)` block matches any type of exception.

Example: Division by Zero and File Not Found

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    try {
        // --- Division by zero ---
        int a = 10, b = 0;
        if (b == 0)
            throw "Division by zero error!";

        cout << "Result: " << (a / b) << endl;

        // --- File not found ---
        ifstream file("data.txt");
        if (!file)
            throw 404;

        cout << "File opened successfully!" << endl;
    }
```

```
        catch (const char* msg) {
            cout << "Exception: " << msg << endl;
        }

        catch (int errorCode) {
            cout << "Exception: File not found! Error code = " <<
            errorCode << endl;
        }

        return 0;
    }
```


⚙️ What Happens Here?

- The program enters the `try` block and starts executing code **sequentially**.
- When it reaches this line:

cpp

Copy

Edit

```
throw "Division by zero error!";
```

it throws an exception immediately.

- Control is then transferred to the matching `catch` block, which handles the `"Division by zero"` error.

🛑 What about the next part (file opening)?

It is never executed because:

- Once an exception is thrown inside a `try` block, the rest of the code **inside that block is skipped**.
- The program **jumps to the first matching** `catch` block, handles the exception, and then **exits the** `try-catch` structure.

How to Handle Both Exceptions Separately?

```
// First exception handling
try {
    if (b == 0)
        throw "Division by zero error!";
}
catch (const char* msg) {
    cout << "Exception: " << msg << endl;
}

// Second exception handling
```

```
try {
    ifstream file("data.txt");
    if (!file)
        throw 404;
}
catch (int errorCode) {
    cout << "Exception: File not found! Error code = " <<
    errorCode << endl;
}
```

Rethrowing an Exception

Rethrowing means **throwing an exception again after catching it**, typically to allow another handler (possibly at a higher level in the call stack) to handle it appropriately.

It is useful when:

- A function catches an exception but **cannot fully handle** it.
- We want to **log, modify, or partially handle** the exception and then let someone else handle it further.

Syntax for Rethrowing

```
try {  
    // Code that might throw an exception  
}  
catch (const std::exception &e) {  
    // Do something (e.g., logging)  
    throw; // rethrows the same exception  
}
```

Key Points About Rethrowing

1. **Preserves Exception Type** : When you use `throw;`, the original exception type is preserved. This allows higher-level handlers to process the exception appropriately.
2. **Modifying Exceptions on Rethrow** : If you want to modify the exception before rethrowing it, you can throw a new exception instead of using `throw;`. For example:

cpp

Copy

```
1 throw runtime_error("Modified error message");
```

3. **Avoid Infinite Loops** : Be cautious when rethrowing exceptions within nested `try-catch` blocks to avoid infinite loops.

Example: Rethrow

```
#include <iostream>
#include <string>
using namespace std;

void lowLevelFunction() {
    try {
        throw runtime_error("An error occurred in lowLevelFunction");
    } catch (const runtime_error& e) {
        cout << "Low-level function caught exception: " << e.what() << endl;
        cout << "Logging the error..." << endl;
        throw; // Rethrow the exception
    }
}

void highLevelFunction() {
    try {
        lowLevelFunction();
    } catch (const runtime_error& e) {
        cout << "High-level function caught exception: " << e.what() << endl;
        cout << "Taking corrective action..." << endl;
        throw;
    }
}

int main() {
    try {
        highLevelFunction();
    }
    catch (const std::exception &e) {
        std::cout << "main caught exception: " << e.what() << std::endl;
    }
}
```

```
catch (...) {
    cout << "Main function caught an unknown exception." << endl;
}
return 0;
}
```

Stack Unwinding

Stack Unwinding refers to the process by which C++ **destroys all local objects** in reverse order of their creation (i.e., LIFO – Last In, First Out) **when an exception is thrown but not caught in the current function.**

✓ It ensures that **destructors** of all local objects are called properly, thus **preventing resource leaks.**

Why is it Important?

- Helps **release memory**, file handles, and other resources.
- Ensures **clean program termination** or recovery when exceptions occur.
- Enables **RAII (Resource Acquisition Is Initialization)** to function properly.

How It Works

When an exception is thrown:

1. The runtime looks for a suitable catch block.
2. On the way out (back through the call stack), **local objects in each function frame are destroyed.**
3. This continues until the matching catch block is found or the program terminates.

Example

```
#include <iostream>
using namespace std;

class Demo {
    string name;
public:
    Demo(string n) : name(n) {
        cout << "Constructor: " << name << endl;
    }
    ~Demo() {
        cout << "Destructor: " << name << endl;
    }
};

void test() {
    Demo d1("Local Object in test()");
    throw runtime_error("Exception in test()");
}

int main() {
    try {
        Demo d2("Local Object in main()");
```

```
        test();
    } catch (exception &e) {
        cout << "Caught Exception: " << e.what() << endl;
    }
    return 0;
}
```

Output:

```
Constructor: Local Object in main()
Constructor: Local Object in test()
Destructor: Local Object in test()
Caught Exception: Exception in test()
Destructor: Local Object in main()
```


Key Points

- **Destructors are called** for all local objects when stack unwinding occurs.
- If you use raw pointers, remember: destructors won't be called unless **smart pointers or RAI** is used.
- If a destructor **throws an exception during stack unwinding**, it causes `std::terminate()`.

Exception as Class

What Does It Mean?

In C++, exceptions are not limited to primitive data types like int or char. You can also **throw and catch user-defined class objects** as exceptions. This helps in **providing more meaningful and structured information** when an exception occurs.

Why Use Classes for Exceptions?

Using classes for exceptions allows you to:

- Include **custom error messages**
- Add **additional attributes** (e.g., error code, severity)
- Implement a **hierarchy** of exceptions using inheritance
- Create **specific catch blocks** for different exception types

Simple Example: Throwing a Class Object

```
#include <iostream>
using namespace std;

class DivideByZero {
public:
    void showMessage() {
        cout << "Error: Division by zero!" << endl;
    }
};

int main() {
    int a = 10, b = 0;

    try {
        if (b == 0)
            throw DivideByZero();
        cout << "Result = " << a / b << endl;
    } catch (DivideByZero e) {
        e.showMessage();
    }

    return 0;
}
```

Output:

```
Error: Division by zero!
```

More Advanced Example with Constructor and Attributes

```
#include <iostream>
using namespace std;

class MyException {
    string message;
    int code;
public:
    MyException(string msg, int c) : message(msg), code(c) {}
    void display() {
        cout << "Error [" << code << "]: " << message << endl;
    }
};

int main() {
    try {
        throw MyException("Invalid operation", 101);
    } catch (MyException &ex) {
        ex.display();
    }

    return 0;
}
```

Output:

```
Error [101]: Invalid operation
```

Standard Exceptions

What Are Standard Exceptions?

C++ provides a set of **predefined exception classes** in the **Standard Library** under the `<stdexcept>` header. These classes are part of a hierarchy rooted at `std::exception`. They offer **meaningful error types** and messages, making exception handling more informative and standardized.

Base Class: `std::exception`

- All standard exceptions inherit from this class.
- It has a virtual function: `virtual const char* what() const throw();`

This function returns a C-style string describing the exception.

Hierarchy of Standard Exceptions

```
std::exception
├── std::logic_error
│   ├── std::invalid_argument
│   ├── std::domain_error
│   ├── std::length_error
│   └── std::out_of_range
└── std::runtime_error
    ├── std::range_error
    ├── std::overflow_error
    └── std::underflow_error
```

Example: Using std::invalid_argument

```
#include <iostream>
#include <stdexcept>
using namespace std;

int squareRoot(int x) {
    if (x < 0)
        throw invalid_argument("Negative value not allowed");
    return x; // Placeholder
}

int main() {
    try {
        squareRoot(-9);
    } catch (exception &e) {
        cout << "Caught Exception: " << e.what() << endl;
    }

    return 0;
}
```

Example: std::out_of_range

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};

    try {
        cout << v.at(5) << endl; // Will throw out_of_range
    } catch (const out_of_range &e) {
        cout << "Out of Range Exception: " << e.what() << endl;
    }

    return 0;
}
```


Benefits of Using Standard Exceptions

- Clear, predefined structure
- Descriptive messages via `.what()`
- Integrates well with STL containers and algorithms
- Encourages **consistent and maintainable** error handling

Example: Exception in Class Member Function

```
#include <iostream>
#include <stdexcept> // For standard exceptions
using namespace std;

class BankAccount {
private:
    double balance;

public:
    BankAccount(double initial) {
        if (initial < 0)
            throw invalid_argument("Initial balance cannot be negative.");
        balance = initial;
    }

    void withdraw(double amount) {
        if (amount > balance)
            throw runtime_error("Insufficient balance.");
        balance -= amount;
    }

    void showBalance() const {
        cout << "Current Balance: ₹" << balance << endl;
    }
};
```

```
int main() {
    try {
        BankAccount account(1000); // Valid initial balance
        account.showBalance();

        account.withdraw(500); // Valid
        account.showBalance();

        account.withdraw(600); // Will throw exception
    }
    catch (const exception &e) {
        cout << "Exception: " << e.what() << endl;
    }

    return 0;
}
```